Studio Nx

JS 05 — parseLoginHash()

2026-10-12

JS 05 — `parseLoginHash(hash)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	04 `loginUrl()` · 06 `isLoggedIn()`

1. 하는 일

주소의 `#login?next=…`를 읽어서, **모달을 열지 여부(`open`)**와 **로그인한 뒤 갈 곳(`next`)**을 돌려줍니다.

`parseLoginHash('#login?next=%2FmyPage.html')` → `{ open: true, next: '/myPage.html' }`

2. 언제 쓰나

04 `loginUrl()`은 로그인 주소를 만들고, 이 함수는 `home.html`에서 그 주소를 읽습니다. 두 함수는 같은 주소 형식(`#login?next=…`)을 쓰므로 바꿀 때는 함께 바꿔야 합니다.

상황	부르는 곳
`home.html`을 열 때	`home.js`의 `initLoginModalFromHash()`(P1-07)가 부릅니다. `open`이면 로그인 모달을 열고 `next`를 기억합니다
로그인에 성공한 뒤	로그인 폼 처리(P1-07)에서 `next`가 있으면 그곳으로, 없으면 기본 페이지로 보냅니다

주소	모달	로그인 후
`home.html`	열리지 않습니다	—
`home.html#login` (헤더의 Sign In, 가입 완료)	열립니다	기본 페이지
`home.html#login?next=%2FmyPage.html` (`requireLogin`)	열립니다	`/myPage.html`

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 쓰지 않습니다.** `safeNextPath`는 같은 파일에 있고, `URLSearchParams`와 `location`은 브라우저의 기본 기능입니다.

함수

```
export function parseLoginHash(hash = location.hash) {
  if (typeof hash !== 'string' || !hash.startsWith('#login')) return { open: false, next: null };
  const rest = hash.slice('#login'.length);
```

```
  if (rest && !rest.startsWith('?')) return { open: false, next: null };
  const next = new URLSearchParams(rest.slice(1)).get('next');
  return { open: true, next: safeNextPath(next) };
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`hash = location.hash`	기본값입니다. 지금 주소의 `#`부터 끝까지입니다 (예: `'#login?next=%2FmyPage.html'`)
`hash.slice('#login'.length)`	앞의 `#login`(6글자)을 잘라낸 나머지입니다. `''` 또는 `'?next=…'`가 됩니다
`rest && !rest.startsWith('?')`	나머지가 있는데 `?`로 시작하지 않으면, `#loginHelp` 같은 **다른** 해시라는 뜻입니다
`rest.slice(1)`	맨 앞의 `?`를 뗀 `next=%2FmyPage.html`입니다
`new URLSearchParams(…).get('next')`	`이름=값&…`을 읽어서 `next` 값을 꺼냅니다. **인코딩을 자동으로 풀어 줍니다** (`%2F` → `/`, `%3F` → `?`). 없으면 `null`입니다

hash	rest	결과
`'#login'`	`''`	`{ open: true, next: null }`
`'#login?next=%2FmyPage.html'`	`'?next=%2FmyPage.html'`	`{ open: true, next: '/myPage.html' }`
`'#loginHelp'`	`'Help'`	`{ open: false, next: null }`
`'#features'`	—	`{ open: false, next: null }`

4. 흐름

1. `#login`으로 시작하지 않으면 모달을 열지 않습니다.
2. `#login` 뒷부분이 비었거나 `?`로 시작하는지 확인합니다 (`#loginHelp`를 거릅니다).
3. `?` 뒤에서 `next`를 꺼냅니다 (인코딩이 풀립니다).
4. `safeNextPath`로 검사합니다. 위험하면 `next: null`이 되지만 **모달은 엽니다**.

5. 왜 이렇게

선택	이유
해시(`#`)에 둠	서버로 가지 않고, 페이지를 다시 불러오지도 않습니다 (04의 보안 설명)
`URLSearchParams`	`&`, `=`, `%XX`를 직접 자르면 실수하기 쉽습니다. 그래서 브라우저 표준 방식으로 읽습니다
읽을 때도 `safeNextPath`로 검사	주소는 누구나 바꿀 수 있습니다. **만들 때 검사했더라도 읽을 때 다시** 검사합니다
위험한 `next`여도 `open: true`	로그인 자체는 정상이므로, 갈 곳만 버리고 기본 페이지로 보냅니다
결과를 객체 하나로 돌려줌	부르는 쪽이 `const { open, next } = parseLoginHash()` 한 줄로 받을 수 있습니다

6. 시험

입력	결과
`#login`	열리고, `next`는 없습니다
`#login?next=%2FmyPage.html`	열리고, `/myPage.html`입니다
`#login?next=%2FstudioNx.html%3FprojectId%3D3`	열리고, `/studioNx.html?projectId=3`입니다
`#login?next=%2FmyProject.html%3Fq%3Da%26page%3D2`	열리고, `/myProject.html?q=a&page=2`입니다
`#login?next=https%3A%2F%2Fevil.com`, `…%2F%2Fevil.com`	열리지만 `next: null`입니다 (풀면 `https://…`, `//evil.com`이라서 거부합니다)
`#login?foo=1`	열리고, `next: null`입니다
`#loginHelp`, `#features`, `''`	열리지 않습니다
인자 없음 (`location.hash` 사용)	`location.hash`를 읽습니다
`loginUrl()`로 만든 주소를 이 함수로 읽기	원래 경로가 그대로 돌아옵니다 (왕복)